Exploring Next Token Prediction For Optimizing Databases

Yeasir Rayhan
Purdue University
West Lafayette, IN, USA
yrayhan@purdue.edu

Walid G. Aref
Purdue University
West Lafayette, IN, USA
aref@purdue.edu

Abstract

The *Next Token Prediction* paradigm (NTP, for short) lies at the forefront of modern large foundational models that are pre-trained on diverse and large datasets. These models generalize effectively, and have proven to be very successful in Natural Language Processing (NLP). Inspired by the generalization capabilities of Large Language Models (LLMs), we investigate whether the same NTP paradigm can be applied to DBMS design and optimization tasks. Adopting NTP directly for database optimization is non-trivial due to the fundamental differences between the domains. In this paper, we present a framework, termed Probe and Learn (PoLe), for applying NTP to optimize database systems. PoLe leverages Decision Transformers and hardware-generated tokens to effectively incorporate NTP into database systems. As a proof of concept, we demonstrate PoLe in the context of the index scheduling task over NUMA servers in main-memory database systems. Preliminary results for this scheduling task demonstrate that adopting NTP and PoLe can improve both performance and generalizability.

Keywords

Next Token Prediction; Database Optimization; NUMA Servers; Data Indexing

ACM Reference Format:
Yeasir Rayhan and Walid G. Aref. 2025. Exploring Next Token Prediction For Optimizing Databases . In *International Workshop on Exploiting Artificial Intelligence Techniques for Data Management (aiDM '25), June 22–27, 2025, Berlin, Germany*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3735403.3735999

1 Introduction

Hardware and workload are the two dominant factors that have shaped DBMS research over the past 50 years, and are termed the "Game Changers" in databases [3]. Both factors are rapidly evolving, and are consistently reshaping the design principles and optimization techniques of modern DBMSs. On the hardware front, recent examples include designs and optimizations on GPUs, e.g., [14, 15, 18, 22, 33, 45], Persistent Memory (PMEM), e.g., [10, 49, 56, 57], SSDs, e.g., [21, 29, 44, 48], Compute Express Link (CXL), e.g., [1, 2, 27, 30] and NUMA servers, e.g., [4, 11, 26, 28, 31, 37–40, 43, 46, 51]. Recently, ARM processors have made significant progress in high-performance computing, and are becoming competitive with the x86 processors with high-end servers, e.g., Apple Silicon [54], Amazon Graviton [6], and NVIDIA Grace [34]. On the workload front, the landscape is equally dynamic, and has taken a drastic turn over the recent years, with the rise of the Large Language Model (LLM, for short) workloads. LLM workloads impose distinct computational and memory demands, further pushing the boundaries of DBMS design and optimization [32].

Another game-changing factor is the heterogeneity of both the hardware and the workloads. On the hardware front, this heterogeneity manifests itself in a variety of compute and memory chips. For example, modern servers are equipped with large cores, small cores, or a mixture of both. Larger cores are preferable for serialized execution, whereas parallelization favors smaller cores. Likewise, nowadays the same Integrated Memory Controllers (IMCs, for short) support both DRAM and PMEM. In addition, the heterogeneity leads to variable read and write access latencies across hardware, e.g., in Non Uniform Memory Access (NUMA) servers accessing data that resides in a remote DRAM chip can incur up to 4× more latency than accessing data that resides in the local DRAM chip [12]. Accessing the PMEM can be 3× slower than accessing a local DRAM chip [23]. On the workload front, this heterogeneity stems from the diverse range of applications that DBMSs must support, including OLTP, OLAP, HTAP, and modern ML workloads. Each of these hardware and workload applications has distinct performance characteristics that require customized optimization strategies. Moreover, the resurgence of cloud databases has taken over traditional on-site DBMSs. Amazon Elastic Compute Cloud (EC2) [5] provides as many as `750` instance choices equipped with a diverse range of processors, memory, storage, and networks hardware to fit different workload applications.

To address these challenges, DBMS engines need to generalize across heterogeneous hardware and workload applications without sacrificing performance. We need a complete paradigm shift in how DBMSs are designed; one that embraces hardware and workload heterogeneity to gain the best performance, while being able to generalize and adapt across new hardware architectures and workload applications without manual intervention. Rather than constantly reinventing the wheel to keep pace with the ever-changing hardware and workload landscape, the goal is to distill decades of DBMS knowledge for diverse workloads and hardware into a compact, transferable form that is both generalizable and adaptable.

Large Language Models (LLMs) [13, 16, 35, 41, 42, 47] have proven to be very successful in generalization tasks across multiple fields, including but not limited to, Natural Language Processing (NLP), Computer Vision, and Robotics. Next Token Prediction (NTP) lies at the core of LLMs. It is a foundational concept in modern AI that involves training models to predict the next element in a sequence based on the preceding context. By mastering NTP, LLMs are responsible for a major share of all AI breakthroughs in many tasks, e.g., machine translation, text summarization, and code generation. It is only natural to question how this simple yet

This work is licensed under a Creative Commons Attribution 4.0 International License.
aiDM '25, Berlin, Germany
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1920-2/2025/06
https://doi.org/10.1145/3735403.3735999

powerful NTP paradigm can be applied to DBMS design and optimizations, and whether the NTP Paradigm can help in building data systems that are capable of generalizing across diverse hardware and workload applications while maintaining high performance.

In this paper, we examine the feasibility of applying the Next Token Prediction paradigm (NTP) in the context of database systems. We address the challenges associated with adopting this paradigm for database systems, and identify two building blocks, namely, Decision Transformers (DT, for short) and a new type of tokens that we term, DB-tokens. DB-tokens are hardware profiles that are essential in translating NTP to DBMS optimizations . We propose a general framework, Probe and Learn (PoLe, for short) for database systems to effectively incorporate the NTP paradigm , and evaluate its effectiveness through a case-study on the main-memory index scheduling task. Preliminary results on the main-memory index scheduling demonstrate that the PoLe framework can learn novel scheduling techniques improving the index performance in query throughput by up to 3× .

The rest of this paper proceeds as follows. Section 2 discusses the Next Token Prediction paradigm, and how it is non-trivial to translate the Next Token Paradigm to DBMSs. Section 3 presents the core building blocks to enable the Next Token Prediction paradigm for database systems. Section 4 introduces the PoLe framework. To demonstrate the effectiveness of PoLe, in Sections 5 and 6, we present a case study of applying PoLe in the context of scheduling indexing tasks into NUMA servers and some preliminary results as a proof-of-concept. Finally, Section 7 concludes the paper.

2 Database Systems And The Next Token Prediction Paradigm

Next Token Prediction: At a high level, the next token prediction (NTP, for short) models the probability of the next token in a given sequence based on the past tokens. In Natural Language Processing (NLP), each token is a discrete unit, i.e., a word, a sub-word, or a character. For any sequence $\boldsymbol{\tau}$, let τ_i denote the i-th token, and let $\boldsymbol{\tau_{<i}}$ denote the $i-1$ tokens preceding τ_i. Then, the goal of NTP is to estimate the probability distribution of τ_i, given $\boldsymbol{\tau_{<i}}$, i.e., $\mathbb{P}(\tau_i|\boldsymbol{\tau_{<i}})$. There are two different phases of NTP, i.e., training-time NTP via self-supervision, and inference-time NTP via autoregression. Refer to Figure 1 for an illustration of the NTP paradigm during both these phases.

- **Training-time NTP.** During training, the model is fed the previous $\boldsymbol{\tau_{<i}}$ ground truth tokens to predict the next token, τ_i. The model is provided with the next ground truth token τ_i so that it can maximize the sum of log-probabilities over the entire sequence to better predict τ_i for discrete actions, i.e., $\mathcal{L}_{\text{total}} = -\sum_{i=1}^{T} \log \mathbb{P}(\tau_i \mid \boldsymbol{\tau_{<i}})$. This is termed as *Teacher Forcing* [55]. Notice that, unlike inference, during training the outputs generated by the models are not fed back into itself. As a result, this avoids propagating any compounding error that may result from any misprediction.
- **Inference-time NTP.** During inference, NTP follows an autoregressive process, i.e., the output generated by the model is fed back into itself to generate the next token. For example, given $\boldsymbol{\tau_{<i}}$ tokens, the model generates the next token τ_i that is then fed back into the model to generate the next token τ_{i+1}. This process continues until a stopping criteria is met. This token generation process is inherently probabilistic, meaning that, at each step, the model outputs a probability distribution over possible next-tokens $\mathbb{P}(\tau \mid \boldsymbol{\tau_{<i}})$ using either greedy selection, i.e., $\tau_i = \arg\max_\tau \mathbb{P}(\tau \mid \boldsymbol{\tau_{<i}})$, or sampling, i.e., $\tau_i \sim \mathbb{P}(\tau \mid \boldsymbol{\tau_{<i}})$.

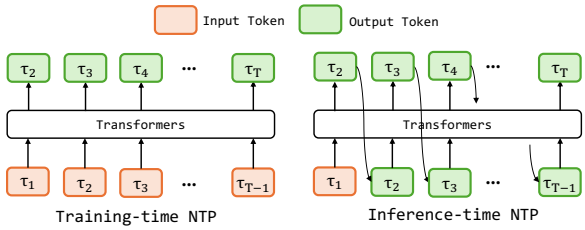

Figure 1: The training and the inference phases of NTP.

Translating The NTP Paradigm into Database Systems: Many database systems problems can be framed as an NTP task. The key question is whether adopting this paradigm effectively aligns with the original objective of the task. For example, consider the Join Order Selection problem (JOS, for short) that involves 4 tables, i.e., A ⋈ B ⋈ C ⋈ D. Framing JOS as an NTP task translates into predicting the next table to join. However, this does not align with the objective of JOS that is to determine the join order that minimizes query execution time, rather than simply determining any join order. One of the reasons behind this discrepancy is the mismatch in objectives between the domain of NLP and that of DBMSs. DBMS optimization tasks are often goal-oriented, and are geared towards improving query performance, scalability, and resource utilization. In contrast, in the era of modern LLMs, NLP tasks often involve generating sequences without explicitly optimizing for any particular objective. This is particularly true during the pre-training phase of LLMs [41], where the goal is to compress a significant amount of world knowledge into the LLM by training on a diverse internet-scale corpus.

Moreover, the notion of a token in NLP remains fixed for a particular tokenizer. NLP tasks additionally benefit from the inherent grammatical rules and the contextual constraints that are consistent throughout the NLP tokens. For example, consider the following sentence: "SIGMOD'25 is going to be held in Germany". The verb follows the subject, reflecting syntactic regularity. Contextually, the mention of Germany also implicitly excludes other locations, e.g., USA. This makes it possible for the NLP tasks to adopt the self-supervised NTP paradigm.

In contrast, the notion of a token in a DBMS can be diverse, e.g., in the Join Order Selection (JOS), the tables can serve as the tokens, whereas in query scheduling decision problems, the queries themselves can serve as the tokens. Both of these entities are fundamentally different. Even in the Join Order Selection problem, two different database instances often have distinct sets of tables, making it harder to learn meaningful representation for the tables, i.e., tokens. Due to this irregular nature of the concept of tokens in DBMSs, a join order, e.g., A ⋈ B ⋈ C ⋈ D does not mean much, i.e., both in terms of grammar and context. Without any additional information about the attributes of the 4 relations, a A ⋈ B ⋈ C ⋈ D, join order does not completely eliminate the possibility of another join order,

e.g., D ⋈ C ⋈ A ⋈ B. Even if both A ⋈ B ⋈ C ⋈ D and D ⋈ C ⋈ A ⋈ B are valid join orders, the crux of JOS lies in identifying the more optimal join order. The table tokens themselves, i.e., A, B, C, D are not enough to distinguish a better join order. This is exactly the reason why it is challenging to adopt the self-supervised NTP paradigm as is for the DBMS optimization tasks, e.g., JOS, because it needs to install knowledge about the DBMS optimization landscape into a large neural network. Thus, as it stands now, the Next Token Prediction (NTP) paradigm does not directly translate to DBMSs. In the next section, we demonstrate the necessary building blocks to adopt the NTP paradigm for optimizing database systems.

3 The Building Blocks

One of the key hindrances that blocks DBMSs from benefiting from the NTP paradigm is the goal-directed nature of its optimization tasks. Besides, the notion of token in DBMSs is versatile, i.e., is context-dependent, and as a whole, lacks the generalization capability. To this end, we need goal-conditioned sequence modeling and a redefinition of the concept of tokenization in the context of DBMSs. In this section, we discuss the building blocks in adopting the NTP paradigm for optimizing database systems.

1. Decision Transformer. Decision Transformer [17] (DT, for short) is a promising framework that provides the support for modeling sequences conditioned on a particular objective. DT can abstract sequence modeling as a Reinforcement Learning problem (RL, for short), where the sequential decision-making is guided by long-term rewards. It ignores traditional RL techniques, i.e., learning value functions or policies through iterative updates. Rather, it treats RL as a supervised sequence modeling problem, i.e., an NTP problem. Precisely, a DT is an auto-regressive sequence model that predicts the next action based on the past observations conditioned on a desired reward. By conditioning on a desired reward, DT is able to generate more effective next-token predictions, contrary to the traditional NTP paradigm that operates without a specific goal. This reward-conditioned nature of DT allows it to align its next-token predictions with optimal decision-making strategies. This aligns well with DBMS tasks with a pre-defined objective, e.g., minimizing query execution time.

Decision Transformers are built on top of the Transformer architecture [50]. It represents a sequence $\boldsymbol{\tau}$ as a combination of 3 types of input tokens, i.e., reward-to-go (RTG, for short), state, and action tokens: $(\hat{r}_0, s_0, a_0, \hat{r}_1, s_1, a_1, \ldots, \hat{r}_{|\boldsymbol{\tau}|}, s_{|\boldsymbol{\tau}|}, a_{|\boldsymbol{\tau}|})$. An RTG token $\hat{r}_t$ denotes the future cumulative reward expected from a given timestep t onward. The initial RTG $\hat{r}_0$ is equal to the reward of the sequence. At Timestep t, DT uses the tokens from the latest K timesteps to predict the next action token a_t. K is the transformer's context length. Then, DT's goal is to learn a policy π_{DT} that estimates the probability distribution of the next action a_t, given $\boldsymbol{s_{\leq t}}$ and $\boldsymbol{\hat{r}_{\leq t}}$, i.e., $\pi_{\mathsf{DT}}(a_t | \boldsymbol{s_{\leq t}}, \boldsymbol{\hat{r}_{\leq t}})$. Analogous to the standard NTP paradigm, both the state and action tokens correspond to word tokens in LLMs.

Training. The Decision Transformer (DT) is trained following the Offline RL paradigm, i.e., it learns a policy from a fixed dataset $\mathcal{D}_{\mathsf{OFF}}$ without online exploration. The dataset $\mathcal{D}_{\mathsf{OFF}}$ comprises a collection of experiences, i.e., state-action transitions, $(\hat{r}_i, s_i, a_i)$, generated by various sources, e.g., heuristic methods, RL agents, or human demonstrations. DT does not interact with the environment, and is entirely reliant on this fixed dataset $\mathcal{D}_{\mathsf{OFF}}$ to learn the best possible policy. Hence, the quality and diversity of the dataset plays a significant role in determining the performance of DT in learning an optimal policy. DT samples a mini-batch of experiences with a sequence length of K from $\mathcal{D}_{\mathsf{OFF}}$, and trains itself to predict the next action token using the standard L2-error (cross-entropy) for continuous (discrete) actions.

Inference. During inference, the model is fed the initial state s_0 and the desired reward, i.e., performance $\hat{r}_0$. Then, DT generates the next action token, $a_0 = \pi_{\mathsf{DT}}(\hat{r}_0, s_0)$. Once generated, a_0 executes to generate the next state token, $s_1 \sim \mathbb{P}(s_1 | s_0, a_0)$ and the next reward-to-go token, $\hat{r}_1 = \hat{r}_0 - R(s_0, a_0)$. Then, DT is fed $\hat{r}_1, s_1$ to generate the next action token prediction a_1. This process repeats until all the action tokens in the sequence are generated.

2. Tokenization. As discussed in Section 2, the discrete word tokens used in language models do not directly translate to database systems. The word tokens in language models serve as linguistic units that remain consistent across all the language tasks, and datasets. In contrast, database systems lack a universal token representation that makes it harder to adopt the NTP paradigm into DBMSs.

Database Tokens: Ideally, the universal token for database systems (DB-token, for short) should be able to generalize across new hardware and workload applications. Besides, it should be computationally inexpensive to compute so that it does not affect the day-to-day database tasks. Moreover, a DB-token should be able to capture additional context because in contrast to language models, these action tokens lack any inherent grammar or structure. These requirements present a significant challenge in identifying an effective internal representation for databases. **We propose to use as DB-tokens, the hardware profiles that are generated from hardware Performance Monitoring Units (PMU, for short).** Hardware profiles are computationally inexpensive to retrieve from the hardware registers, and can generalize across different hardware and workload applications. Moreover, hardware profiles can provide accurate hardware context that the DBMS is running on at any given time. This makes them an ideal candidate for use as DB-tokens in DBMS optimization tasks, as we demonstrate below.

What makes a hardware profile from PMUs stand out in this regard is its capability to provide a real-time update of the hardware and the DBMS components at the highest possible granularity. It goes a long way in mimicking the data distribution and capturing the query workload that have proven to be effective in designing efficient learned DBMSs, e.g., as in [24, 36]. For example, a small region in the data space that is being queried extensively will result in a lot of cache and memory accesses in the cores that handle this data region. The number of cache accesses may further increase, if the data distribution of that region is particularly dense. Likewise, transactional queries yield less cache and memory accesses compared to the analytical queries that yield a large number of executed instructions along with higher cache and memory accesses.

Despite the ongoing evolution in the hardware landscape, the main building blocks of the hardware have remained consistent. At its core, there are compute, memory chips, interconnects and

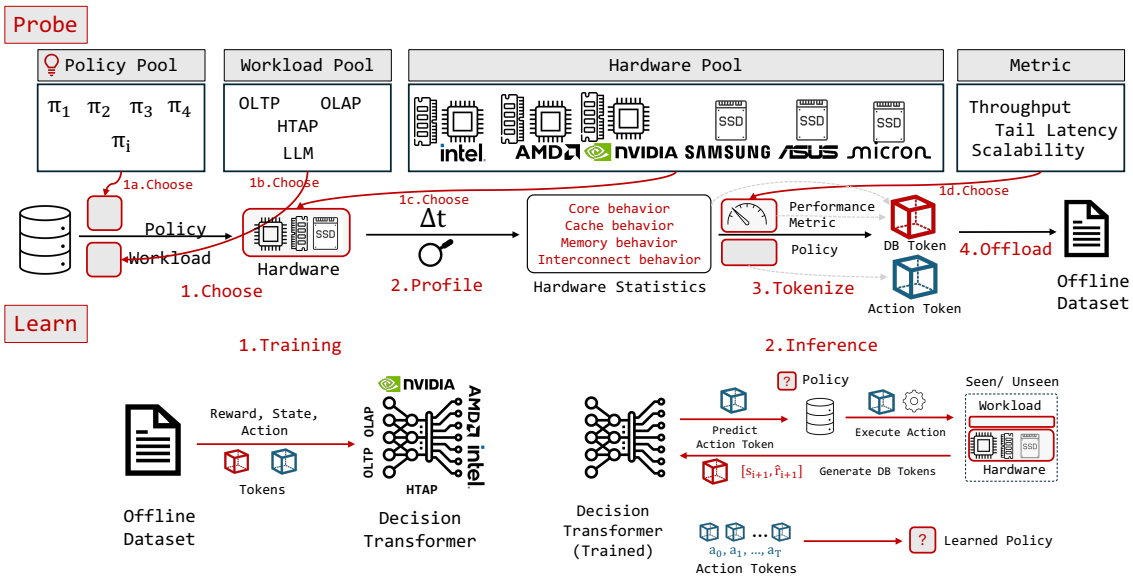

Figure 2: The Probe and Learn (PoLe) framework.

storage. Almost all modern processors provide a number of PMU counters to monitor these critical components. Thus, the PMU counters serve as an excellent fit as representative DB-tokens that capture the database system performance across varying hardware at the finest possible performance granularity. Hence, DB-tokens are an excellent choice as input tokens in an NTP paradigm for optimizing database systems.

Action Tokens: In addition to DB-tokens, there are action tokens to communicate policies across the DBMS and Machine Learning models. These action tokens represent modular units that make up the policy. The action tokens by themselves hold no inherent meaning. It is the DB-tokens that provide the necessary context, allowing a model to understand and learn the intricacies of DBMS optimization and how specific actions impact performance. In the next section, we describe how to connect these two key building blocks, i.e., Decision Transformers and the DB-tokens to develop a framework for adopting the NTP paradigm into database systems.

4 The Probe-and-Learn (PoLe) Framework

To effectively adopt the Next Token Prediction (NTP) paradigm in database systems, we propose a new technique, termed Probe-and-Learn (PoLe, for short). For a given DBMS optimization task, this involves probing the hardware during query execution to generate DB-tokens. These tokens are used to train a DT to learn an optimal policy for the specific optimization task. Refer to Figure 2 for an overview of PoLe.

Probe: The Probe step comprises the following four steps.

- **Choosing.** For a given database optimization task with a set objective, i.e., query throughput, tail latency or scalability, we execute different policies across various hardware configurations and workloads, drawn from a diverse set of policy, hardware, and workload pools. The policy pool includes both optimal and suboptimal strategies that can be randomly sampled or borrowed from existing database literature. The workload pool can comprise a mixture of transactional and analytical workloads. The hardware pool can span servers from different vendors, with varying micro-architectural properties, NUMA configurations, memory technologies, and other distinct characteristics, all of which can significantly impact the performance and behavior of the DBMS.
- **Profiling.** During query execution, we periodically profile the hardware to capture the behavior of crucial hardware components using hardware statistics, e.g., the number of instructions executed, L1-d, L1-i, L2, Last Level Cache (LLC, for short) accesses and misses, branch mispredictions, the number of local and remote memory accesses and misses, etc. Depending on the DBMS optimization task, the granularity of profiling can vary. For example, for scheduling, profiling can be performed at the query level. In contrast, the join order selection and the data partitioning task requires profiling at the level of query operators and data partitions, respectively. Similarly, different optimization tasks can focus on different hardware statistics. For example, NUMA-aware optimizations can focus on local and remote memory accesses to capture the data locality. In contrast, the join order selection task can benefit from the write statistics as they reflect the size of the materialized intermediate results.
- **Tokenization.** Alongside the hardware profiles, we monitor the desired performance metric of the database system, e.g., the average query throughput, the average tail latency, scalability, etc. These hardware profiles along with the performance metric serve as the DB-tokens. These DB-tokens are analogous to the state and RTG-tokens in DT. Moreover, we tokenize the DBMS policies to generate action tokens.
- **Offloading.** Periodically, we offload the action tokens along with the associated DB-tokens to an offline dataset to minimize

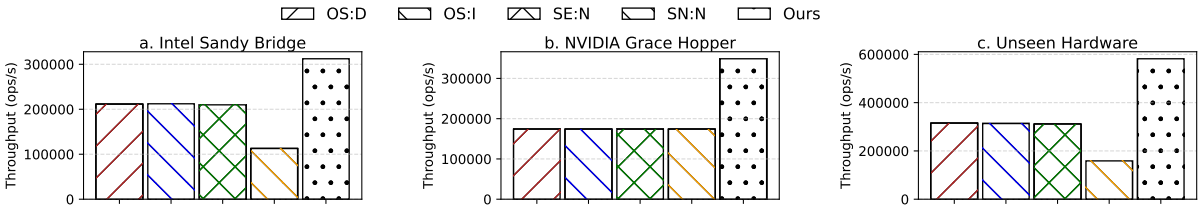

Figure 3: Performance of a main-memory B$^+$-Tree under various scheduling policies.

cache and memory pollution, incurred by this additional data. The offline dataset is provided to the DT during the learning phase.

Learn: The Learn step comprises the following two steps.

- **Training.** We train a Decision Transformer (DT) on the collected offline dataset in a supervised manner following the Teacher Forcing strategy [55]. As the DT is trained on diverse policies across varying hardware and workload configurations, it builds a mental model of the performance landscape of the optimization task at hand. It gains insights into the inherent tradeoffs and the performance improvements across different strategies, hardware and workload via the interaction of action and DB-tokens, that are present in the offline dataset.
- **Inference.** Using the trained DT, PoLe infers a new policy using auto-regression. The process begins by feeding DT with the initial DB-token, i.e., state and the desired objective of the specified optimization task. In turn, DT generates the next action token which is fed back to the DBMS to generate the subsequent DB-token. The DB-token is then fed back into DT to get the next action token. This iterative process continues until the complete policy is generated. Note that, during the inference process, the DT has no explicit knowledge about the hardware that the DBMS is running on, or the workload that the DBMS is executing. Hence, both the hardware, and/or the workload can be unseen. It only relies on the DB-tokens, and the inherent knowledge it has gained by being trained on a wide range of policies, hardware configurations, and workloads to generate the next action token.

5 Case Study: Scheduling Indexing Tasks Over NUMA Servers

To demonstrate the effectiveness of PoLe, we apply it to the main-memory index scheduling task. Modern Non Uniform Memory Access (NUMA, for short) servers are notoriously complex and heterogeneous. Each server can contain more than 1000 cores [9] spread across multiple processor dies. Data access latencies can vary significantly by up to 4× depending on the distance between the core that initiates the data request and the DIMM chip where the data resides. Additionally, concurrently running queries can cause interference by congesting on-chip and off-chip interconnects, as well as saturating hardware queues at caches and memory controllers. Thus, the performance of a DBMS index can vary significantly depending on the physical location of the core where the queries are scheduled, and the DRAM chips where the corresponding data resides.

We formulate main-memory index scheduling as an NTP task and adopt the PoLe framework to solve it. The index is divided into multiple ($\mathcal{I}_c$) index chunks, i.e., a set of index nodes. Given a set of compute cores, the goal is to predict the core on which the queries should be scheduled for the next index chunk. The data placement is implicit, i.e., the data for each chunk resides in the DIMM chip associated with the assigned core. The core ids serve as the action tokens. The clock-cycles, cache, branch misses, local, and remote DRAM accesses along with the query throughput constitute the DB-token. A complete policy includes a sequence of $\mathcal{I}_c$ core ids. The ith core in the sequence represents the core assigned to process the corresponding index slice.

6 Evaluation and Preliminary Results

Experiment settings: We evaluate the learned scheduling policy by PoLe against various OS policies and heuristics. The default OS policy (OS:D) is local allocation, i.e., the OS kernel places an index node on the NUMA node of the requesting CPU. The OS:I policy places the index nodes in an interleaved manner across all the NUMA nodes. For both the Shared Everything NUMA (SE:N) and the Shared Nothing [38] (SN) strategies, nearby index slices are clustered together and are placed in the same NUMA node. For OS:D, OS:I, and SE:N, the OS handles the scheduling. For SN, the scheduling strictly follows the data placement.

We run a YCSB-like benchmark [19] on a main-memory B$^+$-Tree [52, 53] on the YCSB-A workload for 5 minutes. Initially, the index is loaded with 30M records, and grows up to 200M records as more insertions are performed. Each index record has a 64-bit key and a 64-bit value. For these experiments, the B$^+$-Tree is divided into 256 index chunks. The scheduling policy involves allocating the CPU cores for these 256 index chunks.

We evaluate the scheduling policies on the following three machines: Intel Sandy Bridge, NVIDIA Grace Hopper, and Intel Skylake X (Unseen Hardware). The Intel Sandy Bridge is a 4 socket, 4 NUMA Node, 32 (×2) core machine running Ubuntu 22.04 on a CloudLab [20] d820 node. Each socket in the machine is an 8 (×2) core Intel Xeon E5-4620 processor, with an inclusive 64MB L3 cache. Each physical core in the hardware has a 1MB L1-d, 1MB L1-i, and an 8MB L2 cache. The machine is equipped with 128 GB of DDR3 RAM, and the CPU clock frequency is 2.2 GHz. It does not support Sub-NUMA clustering (SNC) or Cluster On Die (COD) technology. The NVIDIA Grace Hopper is a single socket 72 core machine running Ubuntu 22.04 on a CloudLab [20] nvidiagh node. The processor is based on an ARM's Neoverse V2 architecture, where the CPU frequency is 3.1 GHz. The machine is equipped with 480GB

Intel Sandy Bridge (π_l)

32	28	24	44	16	28	48	48	29	29	29	37	25	49	45	51
45	41	21	25	29	51	51	13	33	42	34	45	30	34	27	41
34	45	30	34	29	34	34	30	30	13	36	12	12	12	12	14
26	13	35	20	34	43	39	39	28	26	26	33	26	33	35	50
16	16	16	16	14	18	38	38	47	47	47	50	39	16	21	47
47	28	39	47	47	12	39	39	28	12	28	41	12	16	28	41
31	31	31	31	48	22	13	13	13	23	23	29	29	31	31	31
33	21	45	45	33	44	14	46	46	30	36	36	45	36	17	17
21	43	21	44	51	19	19	15	26	19	29	15	49	49	40	42
30	30	49	49	13	35	17	17	17	39	39	29	29	26	30	29
29	14	29	42	41	26	41	37	37	37	37	30	30	39	13	15
15	39	26	13	17	46	17	34	13	13	13	35	26	31	31	22
22	22	46	46	42	33	33	46	46	46	37	33	33	33	46	37
33	33	37	39	39	22	24	39	21	13	26	26	17	24	24	30
30	16	24	19	19	43	24	24	43	31	43	15	21	21	21	21
50	16	15	31	31	38	43	38	38	38	31	50	38	50	38	38

NVIDIA Grace Hopper (π_l)

13	16	57	52	25	16	27	35	8	17	6	38	40	13	36	4
5	26	56	32	34	43	31	47	6	29	29	15	58	3	46	14
19	48	55	35	53	34	33	39	39	42	39	39	14	53	3	19
45	35	11	49	42	50	55	15	15	15	15	58	53	42	7	19
7	44	42	42	21	54	41	41	41	9	44	21	21	21	21	45
9	9	45	35	51	35	43	43	51	8	39	53	31	31	31	41
20	31	57	46	46	30	55	55	55	30	30	30	36	36	37	36
37	46	37	46	4	37	37	4	4	4	36	58	58	52	43	58
52	41	40	52	24	27	24	40	34	24	52	27	24	24	40	34
56	27	43	27	5	40	19	19	19	9	34	54	10	56	18	5	44
49	38	6	10	30	29	29	47	53	56	50	50	50	50	57	38
48	48	56	38	54	38	48	47	54	48	57	47	57	23	29	10
10	10	54	18	6	6	44	18	28	25	20	5	25	20	28	45
28	28	25	25	7	28	5	20	20	45	11	11	13	7	44	23
7	13	18	13	11	18	11	23	23	23	47	49	9	12	51	22
26	12	3	8	49	3	12	8	3	22	22	22	17	22	12	49

Intel Skylake X (π_l)

31	4	9	7	4	8	7	9	3	3	8	7	6	8	6	5
3	9	6	4	4	3	7	9	52	8	6	4	3	9	6	4
3	4	4	3	9	7	9	30	53	30	6	7	7	6	5	81
80	30	30	30	27	79	6	27	27	27	8	53	8	8	53	28
30	30	54	52	52	52	54	30	8	54	76	8	28	28	33	55
3	28	33	75	8	75	3	3	76	29	29	6	6	52	52	52
52	32	80	32	29	29	9	80	77	9	80	80	5	53	9	80
76	78	5	28	5	32	30	32	31	31	31	51	32	78	51	51
51	31	51	51	51	51	78	76	51	51	32	32	76	32	32	33
33	33	32	78	29	31	80	31	31	33	33	31	80	33	31	33
33	27	77	53	80	27	53	29	80	53	53	29	53	29	28	54
4	28	52	53	52	27	27	27	28	27	30	54	54	79	29	29
4	28	28	79	54	78	54	54	78	54	76	78	76	55	78	76
76	55	55	76	81	81	81	55	5	55	77	5	78	78	5	77
5	77	5	77	81	81	75	75	81	81	75	81	75	81	77	75
56	75	75	79	56	75	79	79	57	57	56	79	57	79	79	57

Figure 4: The scheduling policies learned by PoLe. Each grid corresponds to the learned scheduling policy for the corresponding server for the YCSB-A workload. The top-left grid cell corresponds to the 1st index chunk. The first cell in i-th row corresponds to the $\langle i * 16 + 1 \rangle$-th index chunk. The corresponding text of a grid cell signifies the hardware core where the query for that particular index chunk is scheduled. Cells sharing the same color indicate that the associated index chunks are scheduled on the same NUMA server.

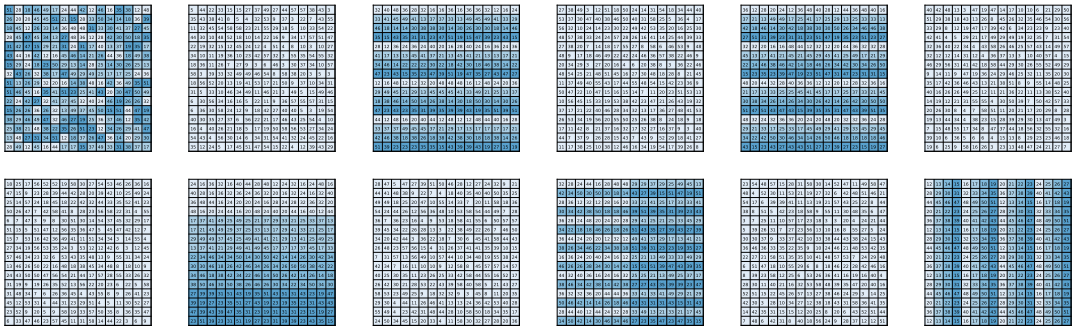

Figure 5: Top performing scheduling policies present in the training dataset for the Intel Sandy Bridge and NVIDIA Grace Hopper servers. Each grid corresponds to a scheduling policy for the corresponding server. The same scheduling policy can be present multiple times in the training dataset for different workloads. Cells sharing the same color indicate that the associated index chunks are scheduled on the same NUMA server.

LPDDR5X RAM. The Skylake-X is a 4 socket, 4 NUMA Node, 92 (×2) core machine running on Ubuntu 22.04. The respective size of L1-d, L1-i, L2, and LLC cache are 3MB, 3MB, 96MB, and 132MB, respectively. The CPU clock frequency is 2.7 GHz.

Preliminary Experimental Results. The DT in the experiment is trained on tokens generated by executing different YCSB workloads, e.g., YCSB-A, YCSB-C, YCSB-E, on a wide range of hardware configurations. Figures 3a and 3b illustrate that PoLe can learn better scheduling policies on Intel Sandy Bridge and NVIDIA Grace Hopper, outperforming the baselines by up to 2.78×. For both these experiments, the workload and the hardware have been encountered during training, i.e., the offline dataset that the DT has been trained on contains experiences from the same hardware and the same workload. Figure 3c illustrates that PoLe can learn a better scheduling policy even for an unseen hardware, i.e., the offline dataset that DT has been trained on does not contain any experience for the particular hardware. It outperforms the baselines by 3× on an unseen Intel Skylake X 4-Socket 4-NUMA-Node machine. Thus, the NTP paradigm not only enhances performance on existing workloads and hardware but also generalizes to optimize index performance in unseen hardware that was not encountered during the training phase.

Figure 4 gives the scheduling policies learned by the DT for the three corresponding servers. Each grid cell corresponds to an index chunk. The number associated with a grid cell corresponds to the hardware core where the query for that particular index chunk has been scheduled. To ensure that the DT does not merely replicate the best scheduling policies from the offline training dataset, we present the top scheduling policies observed during training in Figure 5, and it confirms that the DT does not replicate the scheduling policies from the offline training dataset.

Offline Dataset Preparation Cost: The offline training dataset is generated by executing a diverse set of scheduling policies on a wide range of YCSB workloads on an Intel Sandy Bridge, and an NVIDIA Grace Hopper server. Precisely, the offline dataset contains 445 and 49 scheduling episodes, i.e., samples on the two respective servers. Each episode lasts around 5 minutes. Renting an Intel Sandy Bridge-equivalent Haswell server, i.e., a x1.16xlarge [8] instance on AWS costs approximately $614 ($1.380/hr), while renting an NVIDIA

Grace Hopper server on Lambda cloud [25] costs approximately $73 ($1.49/hr), totaling an approximate cost of $687.[1]

Training Cost: We train PoLe's DT on a dual-socket 64 (×2) core machine running Ubuntu 22.04 on a CloudLab [20] r6525 node. The DT comprises approximately 3.37M trainable parameters. It takes approximately 26 CPU hours to train the DT to reach a training accuracy of over 95%. The inference takes up to 1 CPU minute on the same machine. Renting an equivalent m5a.24xlarge [7] Amazon AWS instance costs approximately $32 ($1.211/hr) to train the DT used in the experiment.

7 Concluding Remarks

In this paper, we lay the foundation for incorporating the Next Token Paradigm in optimizing database systems by proposing a framework that generates hardware tokens to train Decision Transformers in a data-driven framework. Whether all or which database optimization tasks can benefit from the proposed PoLe framework as a means to generalize across diverse hardware and workloads is yet to be seen. However, preliminary results on index scheduling over NUMA servers suggest that PoLe has strong potential in improving performance and on adapting to diverse environments. Building on these early promising results, a key direction for future work is to conduct an extensive investigation into the database optimization landscape to categorize the optimization tasks that can benefit from the Next Token Prediction (NTP) paradigm, and assess to what extent the PoLe framework can provide consistent performance and adaptivity guarantees.

References

[1] Minseon Ahn, Andrew Chang, Donghun Lee, Jongmin Gim, Jungmin Kim, Jaemin Jung, Oliver Rebholz, Vincent Pham, Krishna T. Malladi, and Yang-Seok Ki. 2022. Enabling CXL Memory Expansion for In-Memory Database Management Systems. In *DaMoN*. ACM, 8:1–8:5.

[2] Minseon Ahn, Thomas Willhalm, Norman May, Donghun Lee, Suprasad Mutalik Desai, Daniel Booss, Jungmin Kim, Navneet Singh, Daniel Ritter, and Oliver Rebholz. 2024. An Examination of CXL Memory Use Cases for In-Memory Database Management Systems using SAP HANA. *Proc. VLDB Endow.* 17, 12 (2024), 3827–3840. doi:10.14778/3685800.3685809

[3] Anastasia Ailamaki. 2021. Accelerated Data Management Systems Through Real-Time Specialization. Keynote presented at MICRO.

[4] Martina-Cezara Albutiu, Alfons Kemper, and Thomas Neumann. 2012. Massively Parallel Sort-Merge Joins in Main Memory Multi-Core Database Systems. *VLDB* 5, 10 (2012), 1064–1075. doi:10.14778/2336664.2336678

[5] Amazon. 2024. *Amazon EC2 Instance types.* Retrieved June 24, 2024 from https://aws.amazon.com/ec2/instance-types/

[6] Amazon. 2024. *AWS Graviton Processors.* Retrieved July 28, 2024 from https://aws.amazon.com/ec2/graviton/

[7] Amazon. 2025. *Amazon EC2 m5a.24xlarge instance.* Retrieved May 8, 2025 from https://aws.amazon.com/ec2/instance-types/m5/

[8] Amazon. 2025. *Amazon EC2 x1.16xlarge instance.* Retrieved May 8, 2025 from https://aws.amazon.com/ec2/instance-types/x1/

[9] Anandtech. 2024. *Intel Previews Sierra Forest with 288 E-Cores.* Retrieved July 28, 2024 from https://www.anandtech.com/show/21276/intel-previews-sierra-forest-with-288-e-cores-announces-granite-rapids-d-for-2025-launch-at-mwc-2024

[10] Joy Arulraj, Andrew Pavlo, and Subramanya Dulloor. 2015. Let's Talk About Storage & Recovery Methods for Non-Volatile Memory Database Systems. In *SIGMOD*. ACM, 707–722.

[11] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited. *VLDB* 7, 1 (2013), 85–96. doi:10.14778/2732219.2732227

[12] Tiemo Bang, Norman May, Ilia Petrov, and Carsten Binnig. 2020. The tale of 1000 Cores: an evaluation of concurrency control on real(ly) large multi-socket hardware. In *DaMoN*. ACM, 3:1–3:9. doi:10.1145/3399666.3399910

[13] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, Huazuo Gao, Kaige Gao, Wenjun Gao, Ruiqi Ge, Kang Guan, Daya Guo, Jianzhong Guo, Guangbo Hao, Zhewen Hao, Ying He, Wenjie Hu, Panpan Huang, Erhang Li, Guowei Li, Jiashi Li, Yao Li, Y. K. Li, Wenfeng Liang, Fangyun Lin, Alex X. Liu, Bo Liu, Wen Liu, Xiaodong Liu, Xin Liu, Yiyuan Liu, Haoyu Lu, Shanghao Lu, Fuli Luo, Shirong Ma, Xiaotao Nie, Tian Pei, Yishi Piao, Junjie Qiu, Hui Qu, Tongzheng Ren, Zehui Ren, Chong Ruan, Zhangli Sha, Zhihong Shao, Junxiao Song, Xuecheng Su, Jingxiang Sun, Yaofeng Sun, Minghui Tang, Bingxuan Wang, Peiyi Wang, Shiyu Wang, Yaohui Wang, Yongji Wang, Tong Wu, Y. Wu, Xin Xie, Zhenda Xie, Ziwei Xie, Yiliang Xiong, Hanwei Xu, R. X. Xu, Yanhong Xu, Dejian Yang, Yuxiang You, Shuiping Yu, Xingkai Yu, B. Zhang, Haowei Zhang, Lecong Zhang, Liyue Zhang, Mingchuan Zhang, Minghua Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, Qihao Zhu, and Yuheng Zou. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *CoRR* abs/2401.02954 (2024). doi:10.48550/ARXIV.2401.02954

[14] Nils Boeschen and Carsten Binnig. 2022. GaccO - A GPU-accelerated OLTP DBMS. In *SIGMOD*. ACM, 1003–1016.

[15] Sebastian Breß. 2013. Why it is time for a HyPE: A Hybrid Query Processing Engine for Efficient GPU Coprocessing in DBMS. *Proc. VLDB Endow.* 6, 12 (2013), 1398–1403.

[16] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *NeurIPS*. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[17] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision Transformer: Reinforcement Learning via Sequence Modeling. In *NeurIPS*. 15084–15097.

[18] Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. HetExchange: Encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. *Proc. VLDB Endow.* 12, 5 (2019), 544–556.

[19] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *SoCC*. ACM, 143–154. doi:10.1145/1807128.1807152

[20] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuang-Ching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. 2019. The Design and Operation of CloudLab. In *USENIX*. USENIX Association, 1–14. https://www.usenix.org/conference/atc19/presentation/duplyakin

[21] Gabriel Haas and Viktor Leis. 2023. What Modern NVMe Storage Can Do, And How To Exploit It: High-Performance I/O for High-Performance Storage Engines. *Proc. VLDB Endow.* 16, 9 (2023), 2090–2102. doi:10.14778/3598581.3598584

[22] Bingsheng He and Jeffrey Xu Yu. 2011. High-throughput transaction executions on graphics processors. *Proc. VLDB Endow.* 4, 5 (2011), 314–325.

[23] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amir Saman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dulloor, Jishen Zhao, and Steven Swanson. 2019. Basic Performance Measurements of the Intel Optane DC Persistent Memory Module. *CoRR* abs/1903.05714 (2019).

[24] Tim Kraska, Mohammad Alizadeh, Alex Beutel, Ed H. Chi, Ani Kristo, Guillaume Leclerc, Samuel Madden, Hongzi Mao, and Vikram Nathan. 2019. SageDB: A Learned Database System. In *CIDR*. www.cidrdb.org.

[25] Lambda. 2025. *Renting an NVIDIA Grace Hopper 2000 GPU.* Retrieved May 8, 2025 from https://lambda.ai/nvidia-gh200

[26] Harald Lang, Viktor Leis, Martina-Cezara Albutiu, Thomas Neumann, and Alfons Kemper. 2013. Massively Parallel NUMA-aware Hash Joins. In *IMDM*. 1–12. http://www-db.in.tum.de/other/imdm2013/papers/Lang.pdf

[27] Sangjin Lee, Alberto Lerner, Philippe Bonnet, and Philippe Cudré-Mauroux. 2024. Database Kernels: Seamless Integration of Database Systems and Fast Storage via CXL. In *CIDR*. www.cidrdb.org.

[28] Viktor Leis, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age. In *SIGMOD*. ACM, 743–754.

[29] Viktor Leis, Michael Haubenschild, Alfons Kemper, and Thomas Neumann. 2018. LeanStore: In-Memory Data Management beyond Main Memory. In *ICDE*. IEEE Computer Society, 185–196. doi:10.1109/ICDE.2018.00026

[30] Alberto Lerner and Gustavo Alonso. 2024. CXL and the Return of Scale-Up Database Engines. *Proc. VLDB Endow.* 17, 10 (2024), 2568–2575. doi:10.14778/3675034.3675047

[31] Yinan Li, Ippokratis Pandis, René Müller, Vijayshankar Raman, and Guy M. Lohman. 2013. NUMA-aware algorithms: the case of data shuffling. In *CIDR*. www.cidrdb.org.

[1] The AWS pricing info are collected from: https://instances.vantage.sh/.

[32] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E. Gonzalez, Ion Stoica, and Matei Zaharia. 2024. Optimizing LLM Queries in Relational Workloads. *CoRR* abs/2403.05821 (2024). doi:10.48550/ARXIV.2403.05821 arXiv:2403.05821

[33] Vasilis Mageirakos, Riccardo Mancini, Srinivas Karthik, Bikash Chandra, and Anastasia Ailamaki. 2022. Efficient GPU-accelerated Join Optimization for Complex Queries. In *ICDE*. IEEE, 3190–3193.

[34] NVIDIA. 2024. *NVIDIA Grace CPU Superchip Whitepaper*. Retrieved July 28, 2024 from https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-cpu-superchip

[35] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). doi:10.48550/ARXIV.2303.08774 arXiv:2303.08774

[36] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *CIDR*. www.cidrdb.org.

[37] Danica Porobic, Erietta Liarou, Pinar Tözün, and Anastasia Ailamaki. 2014. ATraPos: Adaptive transaction processing on hardware Islands. In *ICDE*. IEEE Computer Society, 688–699. doi:10.1109/ICDE.2014.6816692

[38] Danica Porobic, Ippokratis Pandis, Miguel Branco, Pinar Tözün, and Anastasia Ailamaki. 2012. OLTP on Hardware Islands. *VLDB* 5, 11 (2012), 1447–1458. doi:10.14778/2350229.2350260

[39] Iraklis Psaroudakis, Tobias Scheuer, Norman May, Abdelkader Sellami, and Anastasia Ailamaki. 2015. Scaling Up Concurrent Main-Memory Column-Store Scans: Towards Adaptive NUMA-aware Data and Task Placement. *VLDB* 8, 12 (2015), 1442–1453.

[40] Iraklis Psaroudakis, Tobias Scheuer, Norman May, Abdelkader Sellami, and Anastasia Ailamaki. 2016. Adaptive NUMA-aware data placement and task scheduling for analytical workloads in main-memory column-stores. *VLDB* 10, 2 (2016), 37–48.

[41] Alec Radford and Karthik Narasimhan. 2018. Improving Language Understanding by Generative Pre-Training.

[42] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners.

[43] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *SIGMOD*. ACM, 1961–1976. doi:10.1145/2882903.2882917

[44] Mehul A. Shah, Stavros Harizopoulos, Janet L. Wiener, and Goetz Graefe. 2008. Fast scans and joins using flash drives. In *DaMoN*. ACM, 17–24.

[45] Panagiotis Sioulas, Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. Hardware-Conscious Hash-Joins on GPUs. In *ICDE*. IEEE, 698–709.

[46] Jens Teubner and René Müller. 2011. How soccer players would do stream joins. In *SIGMOD*. ACM, 625–636. doi:10.1145/1989323.1989389

[47] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023). https://doi.org/10.48550/arXiv.2302.13971

[48] Dimitris Tsirogiannis, Stavros Harizopoulos, Mehul A. Shah, Janet L. Wiener, and Goetz Graefe. 2009. Query processing techniques for solid state drives. In *SIGMOD*. ACM, 59–72.

[49] Alexander van Renen, Viktor Leis, Alfons Kemper, Thomas Neumann, Takushi Hashida, Kazuichi Oe, Yoshiyasu Doi, Lilian Harada, and Mitsuru Sato. 2018. Managing Non-Volatile Memory in Database Systems. In *SIGMOD*. ACM, 1541–1555.

[50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *NeurIPS*. 5998–6008.

[51] Benjamin Wagner, André Kohn, and Thomas Neumann. 2021. Self-Tuning Query Scheduling for Analytical Workloads. In *SIGMOD*. ACM, 1879–1891.

[52] Ziqi Wang. 2017. *Index Micro Benchmarks*. Retrieved October 11, 2024 from https://github.com/wangziqi2016/index-microbench

[53] Ziqi Wang, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. Building a Bw-Tree Takes More Than Just Buzz Words. In *SIGMOD*. ACM, 473–488. doi:10.1145/3183713.3196895

[54] Wikipedia. 2024. *Apple Silicon Processors*. Retrieved July 28, 2024 from https://en.wikipedia.org/wiki/Apple_silicon

[55] Ronald J. Williams and David Zipser. 1989. A Learning Algorithm for Continually Running Fully Recurrent Neural Networks. *Neural Comput.* 1, 2 (1989), 270–280. doi:10.1162/NECO.1989.1.2.270

[56] Xinjing Zhou, Joy Arulraj, Andrew Pavlo, and David E. Cohen. 2021. Spitfire: A Three-Tier Buffer Manager for Volatile and Non-Volatile Memory. In *SIGMOD*. ACM, 2195–2207.

[57] Pengfei Zuo, Yu Hua, and Jie Wu. 2018. Write-Optimized and High-Performance Hashing Index Scheme for Persistent Memory. In *OSDI*. USENIX Association, 461–476.